

Change.org Research Overview

Overview

I developed an end-to-end pipeline for summarizing, embedding, and clustering approximately one million English-language Change.org petitions in order to identify recurring thematic topics. The methodological approach was inspired by [Costello et Al. \(2024\)](#), who demonstrate the value of using large language models (LLMs) to standardize open-ended text prior to embedding and clustering. While our data and research questions differ, we adopted the same core logic, using LLMs as an intermediate representation layer while substantially adapting the implementation to support long-form petition text, large scale, and robust error handling. The pipeline consists of four main stages: AI-based summarization, embedding generation, clustering and model selection, and topic interpretation and consolidation.

AI-Based Summarization of Petitions

The first stage of the pipeline involved converting raw petition text into a concise standardized representation capturing each petition's core claim. This step was necessary because Change.org petitions vary widely in length, structure, and quality, and often contain narrative, rhetorical, or evidentiary material that obscures the central request. Following [Costello et Al. \(2024\)](#), who summarize open-ended responses prior to analysis, we similarly used an LLM to produce a single-sentence representation of each petition's main demand, but adapted this process to much longer and noisier source texts.

Summarization was performed using the OpenAI Chat Completions API with the model gpt-4o-mini. Each petition was summarized into exactly one sentence, framed as an assertion describing what the petitioner is asking for, without normative judgement and without referring to the text as a "petition". Our final prompt went through prompt engineering to ensure that each petition's meaning was still being captured. This was our final prompt used: "You are summarizing Change.org petitions. Summarize the following petition, which describes the author's claim or request, in a single sentence. Do not say that it is a petition, and do not add any normative judgement. Merely accurately describe the content in a way the author who wrote the petition would concur with. Frame it as an assertion. If the petition content is already short, no need to change it very much. If it is quite long and detailed, be sure to capture the core, high-level points, and key words relevant to the topic. Do not focus on the evidence provided for the petition, but merely focus on the basic assertion. Return ONLY strict JSON with keys: petition_id, title, claim."

To control input size and cost, petition text was truncated to a maximum of 500 words prior to summarization, while always including the petition title to preserve context. Only 4.37% of petitions were affected by the truncation. Summarization was executed at scale using the OpenAI Batch API, with the full dataset split across 19 batch jobs to meet file-size and throughput constraints. Batch orchestration, including file upload, batch creation, polling for completion, downloading outputs, and merging results was all handled by one custom runner script, which wraps OpenAI's batch lifecycle methods and supports multiple execution methods (submit, status, download, merge, and submit-and-wait).

Extensive verification and retry logic were required due to the size and corpus. After the initial 10 batches completed, I programmatically identified failed or missing summaries and rebuilt entry batches containing only those petition IDs. These retry batches were then re-submitted so that all summarization errors were re-run together, maximizing coverage. Final batch outputs were parsed line-by-line to extract the claim field from the model's JSON response, joined back to petition IDs via the batch custom_id, and merged into a single JSONL file. We also manually reviewed samples of the generated claims throughout development to confirm that they were representative of the underlying petitions and aligned with our intended abstraction level.

Embedding Generation

Each AI-generated claim was then converted into a numerical vector representation suitable for clustering. Embeddings were generated using the OpenAI model text-embedding-3-small, which produces 1536-dimensional embeddings. Importantly, embeddings were generated from the AI summarized claim text only, not the full petition text, ensuring that the embeddings space reflected the petition's central request rather than supporting narrative or stylistic variation.

Embedding requests were constructed programmatically from the finalized summaries file and submitted via the OpenAI Batch API. Each request used a stable identified format (embed::<petition_id>) to ensure precise alignment between embeddings and petitions during verification and merging. As with summarization, embeddings were generated across 19 batch jobs, followed by a consolidated retry run for all failed embedding requests.

During verification, we identified that approximately 0.01% of petitions (115 petitions) had missing/empty embeddings. Manual inspection revealed that these petitions had empty or degenerate summaries, typically originating from petitions with placeholder titles or non-substantive text (e.g., ".", "ignore", "k", "null", ""). Because embeddings were generated strictly from the claim field, and no meaningful claim existed for these

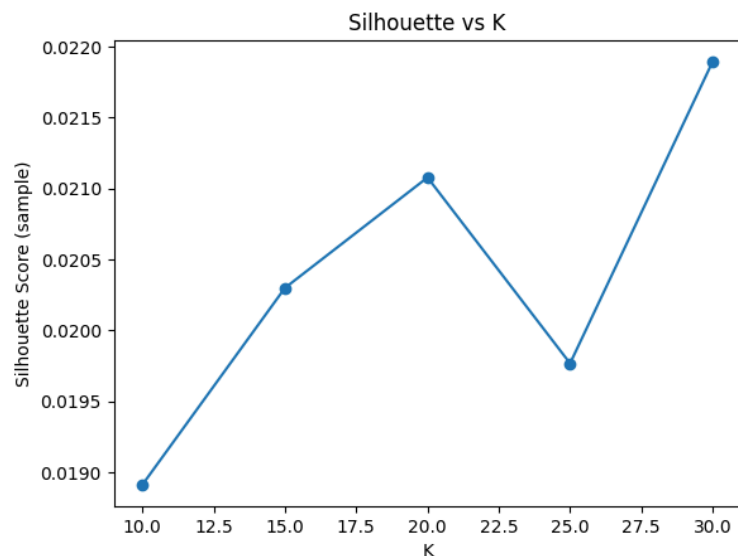
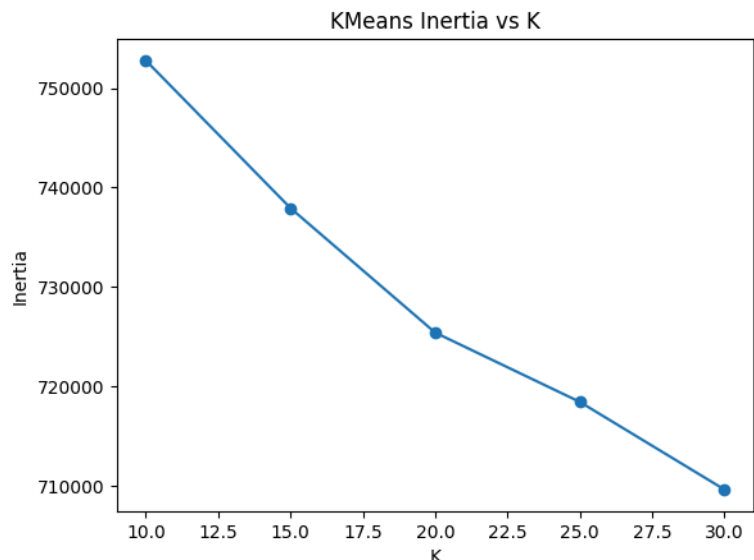
cases, embeddings could not be generated. These cases were therefore treated as unembeddable artifacts of empty source data rather than failures of the embedding process itself. These were not included in the clustering process.

Clustering and Model Selection

Clustering was performed on the full embedding matrix to identify latent thematic structure across petitions. Initial experiments explored density-based clustering (DBSCAN) with cosine distance; however, DBSCAN proved computationally unstable at this scale, frequently collapsing into a single dominant cluster or exhausting available memory.

As a scalable alternative, we used MiniBatchKMeans, which approximates standard K-means by repeatedly updating cluster centroids based on small random batches of observations. By avoiding full-dataset passes at every iteration, this approach enables efficient clustering of very large embedding collections while maintaining results that closely resemble those of standard K-means. Since our final submission was approximately 1,000,000 petitions, MiniBatchKMeans seemed like a great option. Clustering was performed on the full embedding matrix using MiniBatchKMeans with a batch size of 5,000, `random_state=42`, and `n_init = "auto"`, holding these parameters fixed across experiments.

To select the number of clusters, we evaluated $K = \{10, 15, 20, 25, 30\}$ using two complementary diagnostics. We used both the inertia (within-cluster sum of squares) and the silhouette score to determine the optimal K value. The results of these methods can be seen in the graphs to the right.



Because computing silhouette scores on the full dataset is computationally infeasible, silhouette scores were computed on a random sample of 10,000 petitions, sampled without replacement using a fixed random seed, while fitting the clustering model on the full dataset. Inertia decreased monotonically as K increased, while silhouette scores improved up to K=20, declined at K=25, and then increased again at K=30. Considering both metrics, along with the interpretability of resulting clusters, K=20 was selected as the best balance between separation, stability, and thematic coherence.

A final MiniBatchKMeans model with K=20 was fitted to the full embedding matrix, and each petition was assigned a single cluster label. These assignments were saved as a CSV mapping petition_id to cluster_kmeans for downstream analysis.

Topic Interpretation and Consolidation

Following clustering, clusters were interpreted by manually inspecting random samples of petition claims within each cluster. Each cluster was assigned a descriptive topic label based on its dominant theme, with labels determined independently by two annotators, then combined. In a final consolidation step, clusters reflecting closely related subject matter were grouped under broader thematic categories. For example, clusters related to schooling (clusters 0, 9, 14) were consolidated under “School

Cluster Name, Percentage, and Sizes			
Cluster	Name	% of Total	Count
0	School and Education	2.95%	28,438
1	Criminal Justice	4.40%	42,457
2	Pop Culture and Media	4.09%	39,414
3	Conservation and Environment	3.43%	33,107
4	International Politics	7.28%	70,181
5	Consumer Products and Brands	6.27%	60,451
6	Social and Economic Welfare	5.98%	57,724
7	Ban Offensive Content	6.16%	59,427
8	Criminal Justice	6.23%	60,070
9	School and Education	8.18%	78,924
10	Conservation and Environment	3.72%	35,857
11	Pop Culture and Media	7.23%	69,736
12	Conservation and Environment	2.86%	27,608
13	Civil Rights and Diversity	3.97%	38,297
14	School and Education	3.45%	33,248
15	Personal Requests and Inside Jokes	4.44%	42,874
16	Leadership Accountability	4.68%	45,145
17	Community Planning	7.47%	72,018
18	Public Safety	3.85%	37,131
19	Pop Culture and Media	3.36%	32,414

and Education,” entertainment-related clusters (2, 11, and 19) under “Pop Culture and Media,” environmental and animal-related clusters (1 and 8) under “Crime and Policing.”

This manual review and consolidation process mirrors the qualitative validation emphasized by [Costello et Al. \(2024\)](#), ensuring that unsupervised clusters correspond substantially meaningful topics rather than purely statistical artifacts.

Summary

Overall, this semester-long project demonstrates a scalable application of LLM-based summarization, embedding, and clustering to a large corpus of petition text. By combining extensive prompt engineering, batch-based processing, robust error recovery, and human validation, the pipeline produces interpretable topic structure while remaining resilient to data quality issues and computational constraints.